Procesos de Desarrollo de Software

CLASE 5 – SOLID

Agenda de la clase

- SOLID: Principios para un código reusable y extensible.

Principios SOLID

- Una receta para un código excelente:
  - Indentación
  - Nombres de variables claros y concisos
  - 100% cobertura en las pruebas
  - Código mantenible y reusable
- Cualquier código que **no sea mantenible** y que no pueda adaptarse a los contextos cambiantes con facilidad, es un código que solo espera volverse **obsoleto**.

SOLID

- **S** ingle Responsability Principle
- **O** pen/Closed Principle
- **L** iskov Substitution Principle
- **I** nterface Segregation Principle
- **D** ependency Inversion Principle

Single Responsability

- Una clase debe tener solo una razón para cambiar (**responsabilidad**).
- Una responsabilidad tiene que ver con:
  - ¿Qué hace?
  - ¿A quién conoce?
- Se centra en mantener alta la **cohesión**.
- Cada clase debe tener responsabilidad sobre una sola parte de la funcionalidad proporcionada por el software, y esa responsabilidad debe estar completamente encapsulada por la clase.

Ejemplo

```java
public class Calculadora {

    public double calcularTotal(double precio, int cantidad) {
        return precio * cantidad;
    }

    public void imprimirTicket(double total) {
        System.out.println("TOTAL: " + total);
    }
}
```

- **Problema**
- La clase hace **2 cosas**:
  - calcular
  - imprimir
- Tiene **2 razones para cambiar**:
  - cambia el cálculo
  - cambia la forma de imprimir

Solución

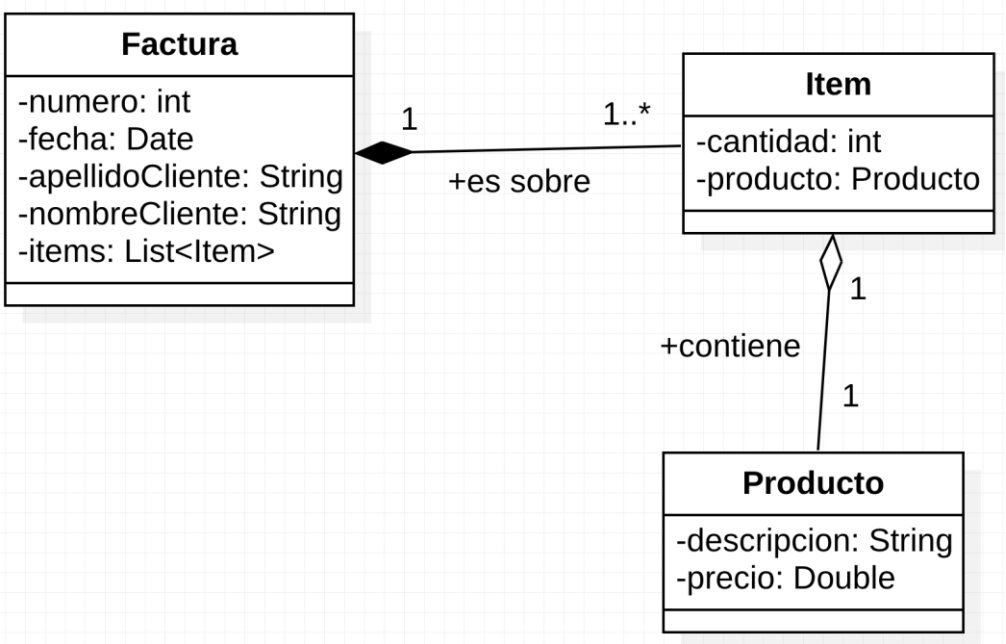
```java
public class Calculadora {

    public double calcularTotal(double precio, int cantidad) {
        return precio * cantidad;
    }
}

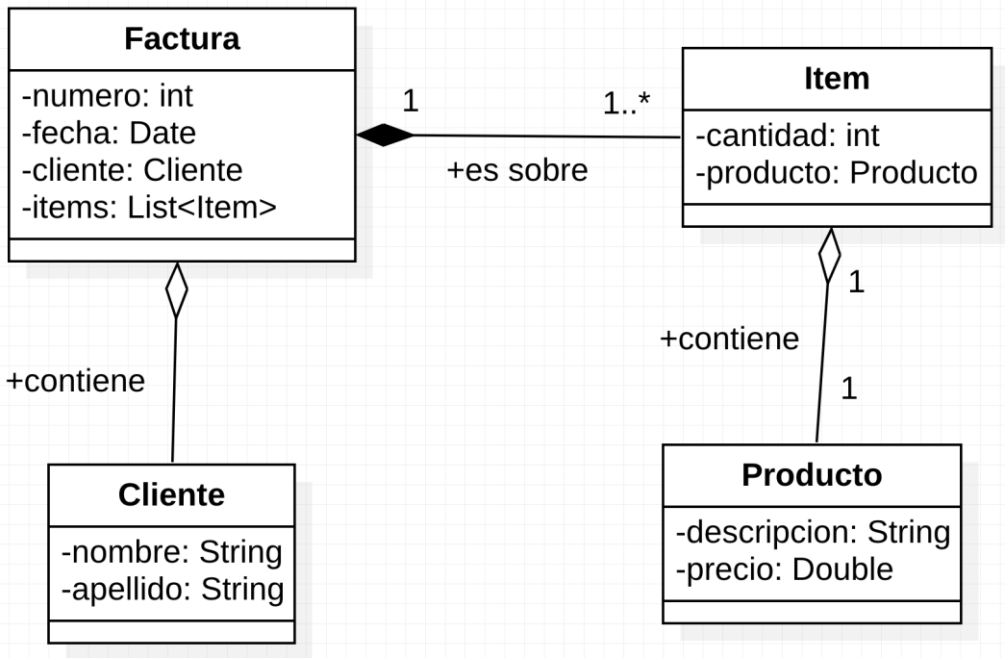
public class Impresora {

    public void imprimir(double total) {
        System.out.println("TOTAL: " + total);
    }
}
```

Práctica


Factura
-numero: int
-fecha: Date
-apellidoCliente: String
-nombreCliente: String
-items: List<Item>
1
1..*
+es sobre
Item
-cantidad: int
-producto: Producto
1
+contiene
1
Producto
-descripcion: String
-precio: Double

Práctica - solución



Open/Closed

- El **comportamiento** de una entidad debe poder ser alterado sin tener que modificar su propio código fuente.
- Una clase **no** se puede **modificar**, pero **si** se puede **extender** haciendo uso de la herencia.
- Se centra en mantener **bajo** el **acoplamiento**.

Ejemplo

```java
public class CalculadoraDescuento {

    public double calcular(double precio, String tipo) {

        if (tipo.equals("NAVIDAD")) {
            return precio * 0.9;
        }

        if (tipo.equals("BLACKFRIDAY")) {
            return precio * 0.7;
        }

        return precio;
    }
}
```

- Cada vez que agregamos un descuento:
  - tenemos que **modificar la clase**
  - rompemos OCP



```java
if (tipo.equals("VERANO")) {
    return precio * 0.8;
}
```

Solución

```java
public interface Descuento {
    double aplicar(double precio);
}

public class DescuentoNavidad implements Descuento {

    public double aplicar(double precio) {
        return precio * 0.9;
    }
}

public class DescuentoBlackFriday implements Descuento {

    public double aplicar(double precio) {
        return precio * 0.7;
    }
}

public class CalculadoraDescuento {

    public double calcular(double precio, Descuento descuento) {
        return descuento.aplicar(precio);
    }
}
```

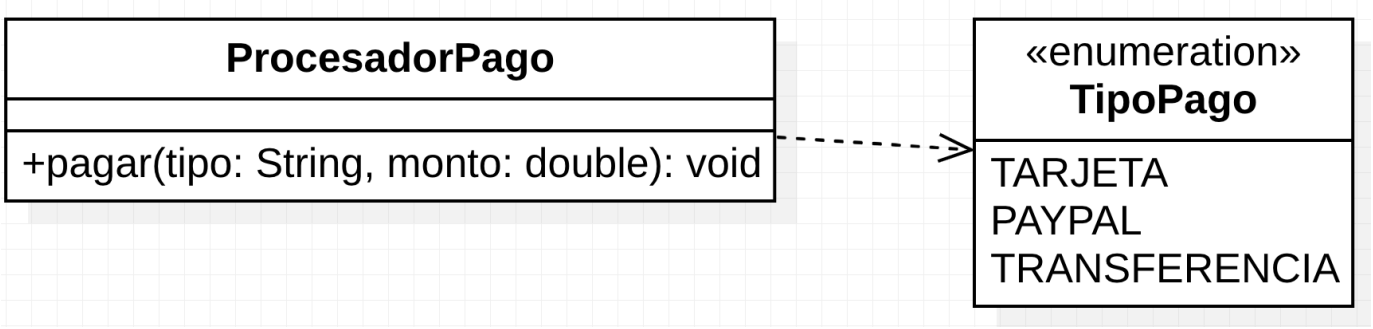
- Ahora agregamos **sin** modificar nada.



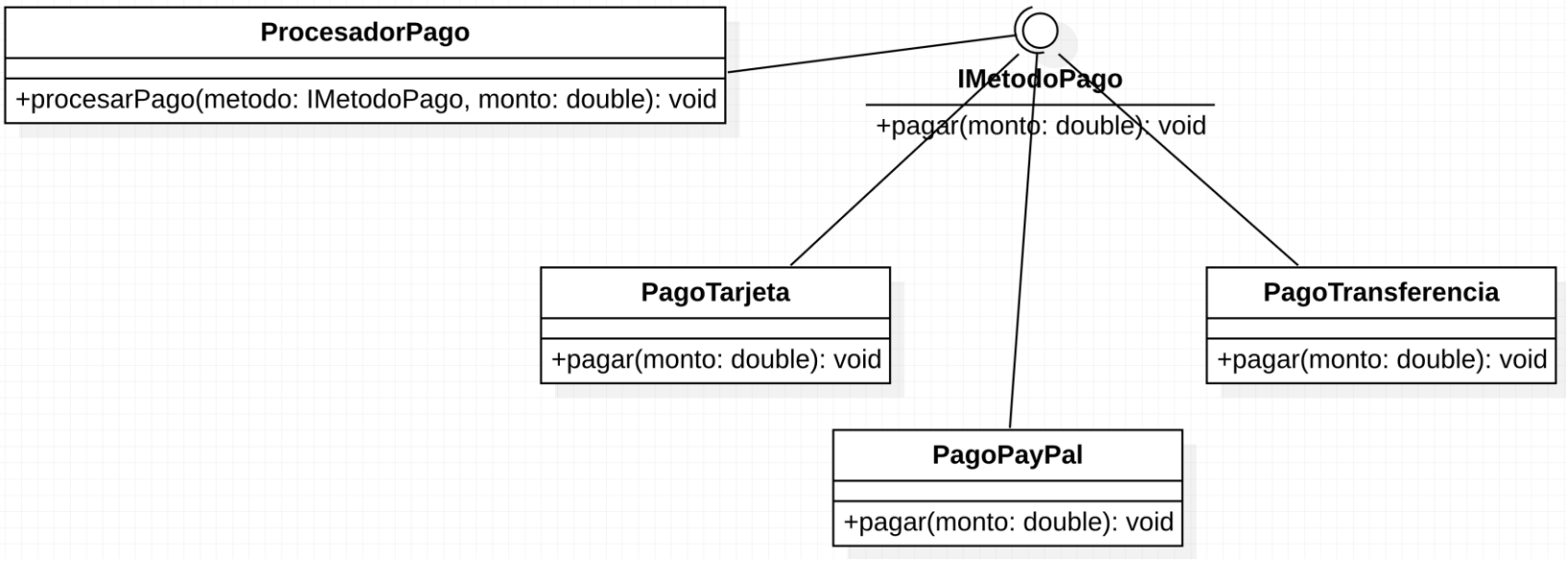
```java
public class DescuentoVerano implements Descuento {

    public double aplicar(double precio) {
        return precio * 0.8;
    }
}
```

Práctica


ProcesadorPago
+pagar(tipo: String, monto: double): void
«enumeration»
TipoPago
TARJETA
PAYPAL
TRANSFERENCIA

Práctica - solución



Liskov Substitution

- **Extensión** del principio **open/closed**.
- Una clase derivada puede ser **reemplazada** por cualquier otra que use su clase base **sin** alterar el correcto funcionamiento de un programa.
- Si una función espera como parámetro una clase base esta puede ser **reemplazada** por cualquier clase derivada.
- Una subclase **no** debe **remover ni modificar comportamiento** de la **clase base**, no debe conocer a los demás subtipos.
- Si un subtipo hace algo que el cliente del supertipo no espera, es una violación al principio.

Ejemplo

```java
class Ave {
    public void volar() {
        System.out.println("Volando...");
    }
}

class Pinguino extends Ave {
    public void volar() {
        throw new RuntimeException("No puedo volar");
    }
}
```

```java
Ave ave = new Pinguino();
ave.volar();
```

- Esperábamos que vuele, pero lanza excepción

Solución

```
class Ave {
    ...
}

interface Volador {
    void volar();
}

class Paloma extends Ave implements Volador {
    public void volar() {
        System.out.println("Volando...");
    }
}

class Pinguino extends Ave {
    ...
}
```

- Ahora nunca "**rompe**".

Ejemplo

```java
public class Rectangulo {
    protected int ancho;
    protected int alto;

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public int getAncho() {
        return ancho;
    }

    public int getAlto() {
        return alto;
    }

    public int calcularArea() {
        return ancho * alto;
    }
}
```

```java
public class Cuadrado extends Rectangulo {

    @Override
    public void setAncho(int ancho) {
        this.ancho = ancho;
        this.alto = ancho;
    }

    @Override
    public void setAlto(int alto) {
        this.alto = alto;
        this.ancho = alto;
    }
}
```

Solución

```
public class CalculadorDeAreas {

    public void imprimirArea(Rectangulo r) {
        r.setAncho(5);
        r.setAlto(4);

        System.out.println("Ancho: " + r.getAncho());
        System.out.println("Alto: " + r.getAlto());
        System.out.println("Área esperada: 20");
        System.out.println("Área real: " + r.calcularArea());
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Rectangulo r = new Cuadrado();

        CalculadorDeAreas calculador = new CalculadorDeAreas();
        calculador.imprimirArea(r);
    }
}
```

- ¿Qué valor arroja el área en verdad?

Interface Segregation

- Los clientes **no** deben estar forzados a **implementar interfaces** que **no usan**.
- Guarda relación con la **cohesión** de las aplicaciones.
- Las clases que **implementen** una **interfaz** o una clase abstracta **no** deberían estar **obligadas** a utilizar partes que **no necesitan**.
- Los clientes no deben estar obligados a implementar y/o a depender de una interfaz que luego no usarán.
- Es mejor tener **interfaces chicas y específicas**, que una interfaz grande con cosas que algunas clases no necesitan.

Ejemplo

```java
public interface MetodoPago {
    void pagar(double monto);
    void devolver(double monto);
    void autenticar();
    void generarCuotas();
}
```

```java
public class PagoTarjeta implements MetodoPago {

    @Override
    public void pagar(double monto) {
        System.out.println("Pagando con tarjeta");
    }

    @Override
    public void devolver(double monto) {
        System.out.println("Devolviendo pago a tarjeta");
    }

    @Override
    public void autenticar() {
        System.out.println("Autenticando tarjeta");
    }

    @Override
    public void generarCuotas() {
        System.out.println("Generando cuotas");
    }
}
```

Ejemplo

```java
public interface MetodoPago {
    void pagar(double monto);
    void devolver(double monto);
    void autenticar();
    void generarCuotas();
}
```

```java
public class PagoTransferencia implements MetodoPago {

    @Override
    public void pagar(double monto) {
        System.out.println("Pagando con transferencia");
    }

    @Override
    public void devolver(double monto) {
        System.out.println("Devolviendo transferencia");
    }

    @Override
    public void autenticar() {
        // no aplica
    }

    @Override
    public void generarCuotas() {
        // no aplica
    }
}
```

- PagoTransferencia está obligada a implementar métodos que **no necesita**.
- Eso genera cosas como:
  - métodos vacíos
  - excepciones
  - comportamiento forzado
  - diseño confuso

Solución

```java
public interface Pagable {
    void pagar(double monto);
}

public interface Reembolsable {
    void devolver(double monto);
}

public interface Autenticable {
    void autenticar();
}

public interface Cuotificable {
    void generarCuotas();
}
```

- Separamos por responsabilidades.

Solución

```java
public class PagoTarjeta implements Pagable, Reembolsable, Autenticable, Cuotificable {

    @Override
    public void pagar(double monto) {
        System.out.println("Pagando con tarjeta");
    }

    @Override
    public void devolver(double monto) {
        System.out.println("Devolviendo pago a tarjeta");
    }

    @Override
    public void autenticar() {
        System.out.println("Autenticando tarjeta");
    }

    @Override
    public void generarCuotas() {
        System.out.println("Generando cuotas");
    }
}
```

```java
public class PagoTransferencia implements Pagable, Reembolsable {

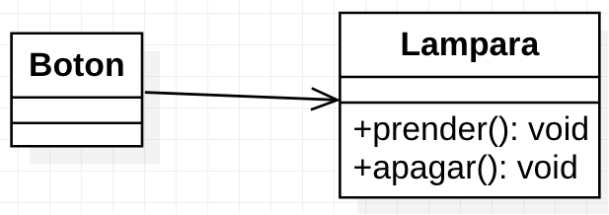
    @Override
    public void pagar(double monto) {
        System.out.println("Pagando con transferencia");
    }

    @Override
    public void devolver(double monto) {
        System.out.println("Devolviendo transferencia");
    }
}
```

Dependency Inversion

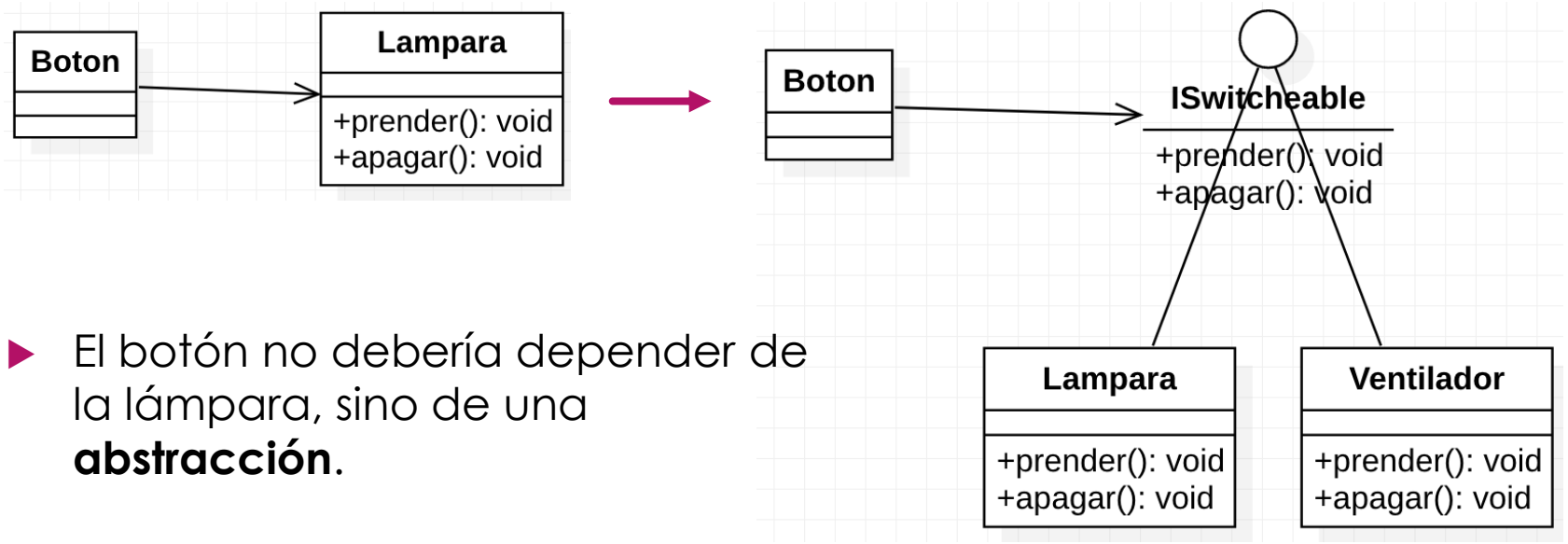
- Se utiliza para **desacoplar** módulos de software.
- Las clases de **alto nivel no** deberían **depender** de las **clases** de **bajo nivel**. Ambas deberían depender de las **abstracciones**.
- Las abstracciones no deberían depender de los detalles. Los detalles deberían depender de las abstracciones.
- Al diseñar la interacción entre un módulo de alto nivel y uno de bajo nivel, la **interacción** debe considerarse como una interacción **abstracta** entre ellos.

Ejemplo



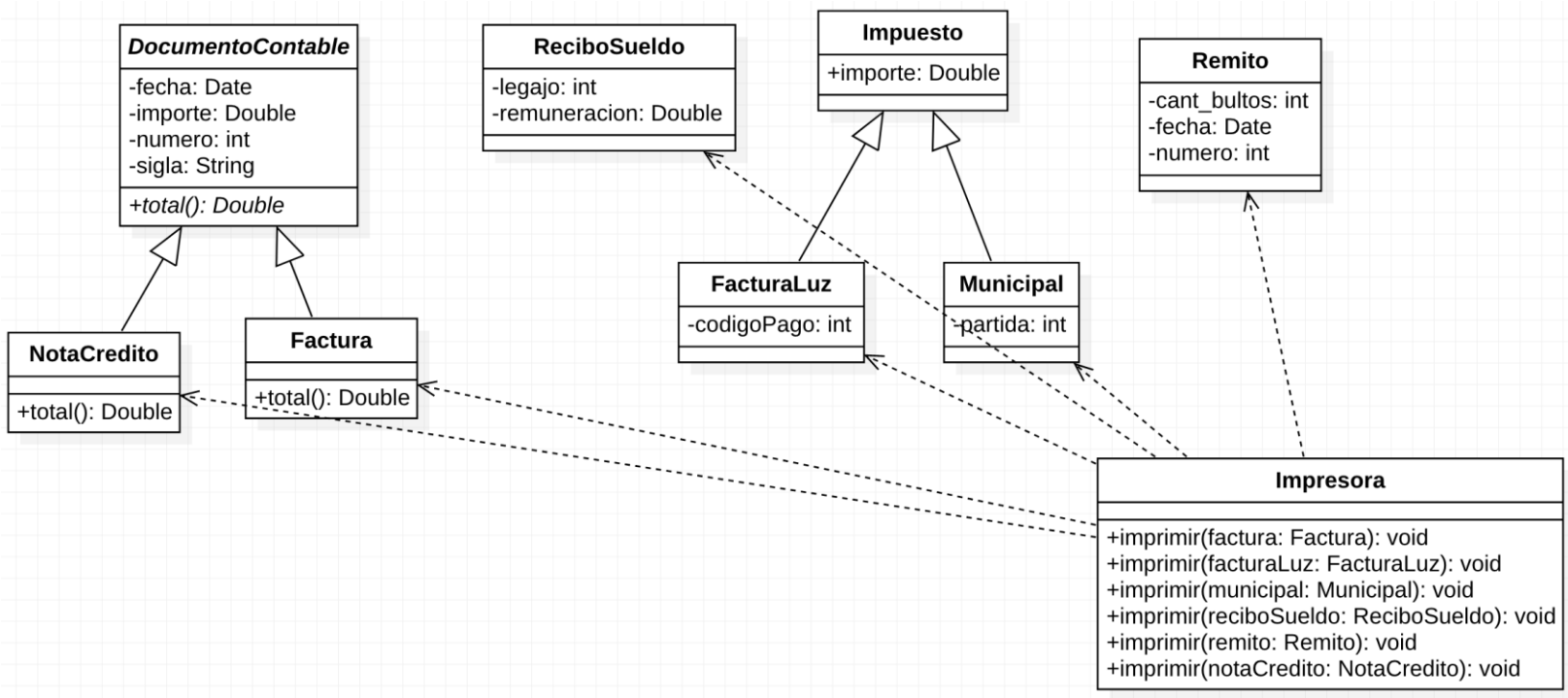
- Un botón es capaz de detectar si el usuario desea prender o apagar una lámpara. Una vez que lo detecta, procede a prenderla o apagarla según corresponda.

Ejemplo

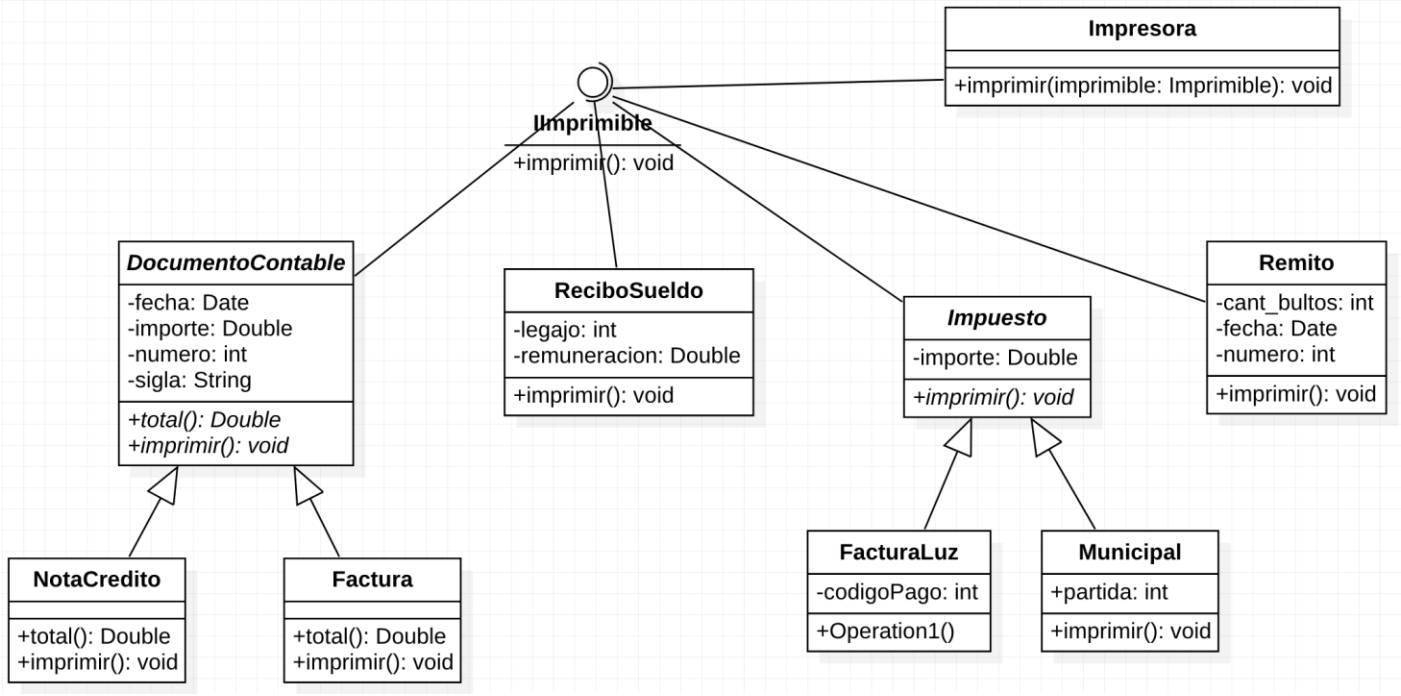


- El botón no debería depender de la lámpara, sino de una **abstracción**.

Ejemplo


DocumentoContable
-fecha: Date
-importe: Double
-numero: int
-sigla: String
+total(): Double
ReciboSueldo
-legajo: int
-remuneracion: Double
Impuesto
+importe: Double
Remito
-cant_bultos: int
-fecha: Date
-numero: int
FacturaLuz
-codigoPago: int
Municipal
-partida: int
NotaCredito
+total(): Double
Factura
+total(): Double
Impresora
+imprimir(factura: Factura): void
+imprimir(facturaLuz: FacturaLuz): void
+imprimir(municipal: Municipal): void
+imprimir(reciboSueldo: ReciboSueldo): void
+imprimir(remito: Remito): void
+imprimir(notaCredito: NotaCredito): void

Solución


Impresora
+imprimir(imprimible: Imprimible): void
Imprimible
+imprimir(): void
DocumentoContable
-fecha: Date
-importe: Double
-numero: int
-sigla: String
+total(): Double
+imprimir(): void
ReciboSueldo
-legajo: int
-remuneracion: Double
+imprimir(): void
Impuesto
-importe: Double
+imprimir(): void
Remito
-cant_bultos: int
-fecha: Date
-numero: int
+imprimir(): void
NotaCredito
+total(): Double
+imprimir(): void
Factura
+total(): Double
+imprimir(): void
FacturaLuz
-codigoPago: int
+Operation1()
Municipal
+partida: int
+imprimir(): void

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

